Medical Portal — Interview Cheat Sheet

Frontend-first talking points with the Flask/PostgreSQL architecture behind them. Based on the code in medical-portal and python_server on 13 October 2026. This is a fictional-data portfolio project, not clinical software.

1. The 60-second project pitch

"I built a medical imaging workflow demo to learn modern React while applying backend concepts I already knew. A React/TypeScript UI talks to a Flask API backed by PostgreSQL. Users sign in through an HttpOnly-cookie JWT; admins manage users, assignments, templates and TAT; radiologists see only assigned studies, draft structured reports, request second opinions and submit to QA; QA validates, and a signer completes the report. Each user belongs to a tenant, and the API filters data by tenant and role. I used fictional data and kept DICOM viewing out of scope so the project stays focused on UI and workflow design."

My contribution story: be precise about what you designed, changed, debugged and verified. If asked about AI, say it helped generate code, but you reviewed behavior, traced the data flow, corrected bugs and can explain each decision. Do not imply you wrote every line unaided.

2. Architecture: follow one request

Browser UI → Vite development server → /api proxy → Flask route → authentication/authorization → SQLAlchemy/PostgreSQL → JSON response → TanStack Query cache → React render.

The proxy target comes from BACKEND_URL in the frontend .env, defaulting to http://127.0.0.1:5000. The frontend calls relative /api URLs with credentials: 'include'. Vite's proxy is **development only**: a production host must serve /api through a reverse proxy or an explicitly configured backend origin and cookie/CORS policy.

Code map: src/main.tsx creates the React root and QueryClient; src/App.tsx owns navigation, auth state handoff and major queries; src/lib/api.ts centralizes typed fetch calls; src/features/* contains screens/components. Backend src/database_app.py creates Flask and core routes/auth helpers; src/models.py defines tables; src/reporting.py, src/report_templates.py, src/tat.py, src/tenancy.py, and src/seed.py separate workflow concerns. Tests live in backend tests/.

3. Frontend stack: why these choices?

Q: Why React?
A: The portal has shared state and many interactive views: filters, dialogs, forms, worklists and role-specific actions. Component composition lets those evolve independently. For a simple static page, React would be unnecessary.

Q: Why TypeScript?
A: API payloads and component props are easy to mismatch. Interfaces such as Patient, ReportSections and WorklistQuery catch mistakes at build time. Types do not validate server responses at runtime; production-hardening could add schema validation.

Q: Why Vite rather than Create React App or Webpack?
A: Vite gives a small React/TypeScript setup, quick dev feedback and a simple production build. This project does not need a custom Webpack pipeline or framework-level server rendering. Vite is a build/dev tool, not the runtime backend.

Q: Why not Next.js?
A: This is an authenticated, client-heavy dashboard backed by an existing Flask API. Server-side rendering and Next.js routing would add complexity without much value for this demo. If SEO, public pages or server components became requirements, I would reconsider.

Q: What does npm run build do?
A: Here it runs tsc -b first, then vite build. Type errors stop the build before bundling. The output is static frontend assets; it does not package Flask or PostgreSQL.

Q: Why TanStack Query instead of fetching in every useEffect?
A: It provides query keys, caching, loading/error states and invalidation after mutations. It handles **server state**; useState handles temporary UI state. It is not a substitute for backend authorization.

Q: Why not Redux?
A: Most shared data comes from the server and already fits Query. The remaining state is local to App or a component. If many unrelated components needed complex shared client state, I would reassess.

Q: Why Tailwind CSS?
A: Utility classes made dark/light variants and responsive layouts quick to iterate. Trade-off: JSX can become dense and repeated styles need extraction. There is also leftover starter CSS in App.css that I would clean up.

4. React state, rendering and data fetching

Q: What lives in useState?
A: Current section, search input, filters, selected patient/study, open dialogs, form values and theme presentation. These are transient UI concerns.

Q: What lives in TanStack Query?
A: The current user, patient pages, dashboard summary, details, calendar, worklist, templates and preferences. Query keys encode filter/page values, so different requests have distinct cache entries.

Q: What does enabled do?
A: It stops a query until prerequisites exist, e.g. do not fetch patient details without a selected ID or protected lists before a user is known.

Q: Why invalidate after a mutation?
A: Saving a patient, changing scan status or scheduling a study can change several screens. Invalidating relevant query keys asks Query to refetch and prevents stale dashboard/detail/list data. A more mature design could centralize these invalidation rules.

Q: Why debounce search?
A: App.tsx and the worklist wait about 300 ms after typing before changing the query key. It reduces API calls during rapid typing. This is distinct from pagination; changing filters resets the page to 1.

Q: Why server-side pagination for patients/worklist?
A: The API returns a page plus total/pagination metadata, avoiding downloading every row. The MIS page currently fetches matching rows and paginates locally; at scale I would paginate/aggregate in SQL.

Q: What are controlled inputs?
A: Their value comes from React state and onChange updates it. The login and report forms use this pattern, making validation, unsaved-change warnings and explicit save possible.

Q: Why key on list rows?
A: Stable IDs let React preserve identity across reorders and updates. Patient/study IDs are better than array indexes.

Q: How does login move to the dashboard immediately?
A: The login form awaits the API, then App writes the returned user into the ['current-user'] query cache, clears prior-user queries and resets local selection/filter state. On refresh, /auth/me restores the session from the cookie.

Q: What does React StrictMode do here?
A: It helps expose development-only lifecycle/effect problems; it is not an auth or caching feature. Query keys and effect cleanup should remain correct under repeated render/effect checks.

Q: How are loading and errors represented?
A: Queries expose loading/error/fetching states; mutations show pending/error UI. apiFetch parses the API envelope and throws ApiError for non-2xx responses. A 401 from /auth/me becomes a logged-out state.

Q: Why is App.tsx a potential refactor target?
A: It holds many screens, filters and mutations and conditionally renders sections. I would move navigation/layout into dedicated components, add URL routing for deep links and feature hooks for queries/mutations.

Q: How is the theme implemented?
A: App applies a .dark wrapper. Tailwind's custom dark variant in src/index.css responds to it; the preference is fetched/saved through Flask. The login page deliberately remains a fixed dark design.

5. Authentication and security: precise answers

Q: Where is the JWT stored?
A: Flask sets it as an HttpOnly cookie. The frontend never reads or stores it in localStorage. apiFetch sends credentials with each request; /auth/me restores the session after refresh.

Q: How does logout work?
A: The frontend calls /api/auth/logout; Flask clears the cookie. The UI clears cached user-specific queries and returns to login. The JWT is not individually revoked server-side, so a copied unexpired token would remain valid until expiration unless additional revocation were built.

Q: Why did login once succeed but the UI stay on login until refresh?
A: The cookie existed, so /auth/me worked after refresh, but React's current-user cache had not been updated in the same render cycle. Writing the returned user to the query cache in onLogin fixed that UI state transition.

Q: What is the role of CORS versus the Vite proxy?
A: CORS governs cross-origin browser requests. The Vite proxy keeps development API calls on the frontend origin and forwards them to Flask. CORS is still configured for the two local dev origins, but CORS alone does not solve cross-site cookie attributes.

Q: Does hiding admin buttons secure admin APIs?
A: No. React conditionally shows controls for usability. Flask routes use require_auth, require_admin and permission checks; study actions also verify assignment and workflow state. A direct API call must be denied if unauthorized.

Q: What security work is still missing?
A: This is a demo, not a production PHI system. Before real data: HTTPS and Secure cookies, CSRF strategy, rate limiting, secret management, robust audit retention, secure deployment, access review, threat testing, backups and compliance assessment. SameSite is a useful cookie control, not a complete CSRF design.

6. Frontend features and interview follow-ups

Q: How does patient search work?
A: React sends search, scan status, report status, modality, date range, page and page size as query parameters. Flask filters a tenant/role-scoped SQL query and returns data plus total. The client does not fetch all rows and filter them for the main list.

Q: How does the calendar handle dates?
A: The UI builds local month boundaries and converts them to UTC ISO timestamps for the API. The database stores UTC; the browser displays local time with Date/Intl. Exclusive next-day end bounds avoid dropping events late on the selected final day.

Q: How does the radiologist worklist stay private?
A: The UI requests GET /api/reporting/worklist with role/stage/modality/search/sort/page. The backend takes the user from authentication, not from a caller-supplied user ID, and filters by tenant and assignment/reviewer ID.

Q: How is report editing safe from accidental replacement?
A: Applying a template changes local editor sections only, prompts before replacing non-empty content, and requires an explicit Save draft. The editor tracks saved versus current sections and warns before discarding unsaved changes. Backend validation still owns the rules.

Q: Why structured sections rather than one textarea?
A: Clinical history, technique, findings and impression can be validated and rendered consistently. Legacy free-text drafts remain readable and can be moved into structured fields. For this demo, JSON sections avoid a larger schema; a full product may need versioned report documents.

Q: Why not add a DICOM viewer?
A: The project is about frontend workflow and has no DICOM storage/viewer. Orthanc/viewer integration would be a separate infrastructure and regulatory concern, not a feature to claim here.

Q: What would you improve in accessibility?
A: Several controls have labels, status/error roles, keyboard activation and responsive layouts. I would add automated axe checks, focus trapping/return for dialogs, stronger keyboard tests and contrast review across both themes.

7. Flask, PostgreSQL and data model

Q: Why Flask?
A: It is a small API framework I knew, so I could focus the learning effort on React while still expressing clear routes, decorators and domain rules. SQLAlchemy handles models/queries; PostgreSQL is the development database.

Q: What are the key tables?
A: tenants; users, roles, permissions and join tables; user_preferences; patients; scans; appointments; notifications; audit_events; report_templates; and per-tenant/per-modality tenant_assignment_cursors.

Q: Why separate Patient and Scan?
A: A patient may have many studies, each with modality, dates, assignment, report state and TAT. Patient demographics should not be duplicated per scan.

Q: How is a tenant determined at login?
A: Login uses a globally unique, case-insensitive email; the backend finds the user and derives their tenant. It never trusts a tenant name typed into the login form. The JWT contains user and tenant IDs, and each authenticated request checks them against the active user/tenant.

Q: Is multi-tenancy just a tenant_id column?
A: No. Routes filter by authenticated tenant; direct ID lookups return 404 for another tenant; assigned reporting users get narrower study visibility; PostgreSQL migrations add matching-tenant foreign-key constraints. Row-level security and an independent production isolation audit are not implemented.

Q: Roles versus permissions?
A: Users have roles; roles map to permission codes. Some endpoints check a permission (e.g. patient write), while admin/report workflow endpoints check role and assignment/state explicitly. Roles are not merely frontend labels.

Q: How does validation work?
A: Routes validate JSON shape, types, dates, range limits, allowed modalities and workflow transitions and return structured errors with HTTP status codes. TypeScript types do not replace server validation.

Q: What do 401, 403, 404 and 409 mean here?
A: 401: not authenticated/invalid cookie. 403: authenticated but lacks role/assignment. 404: absent or intentionally hidden cross-tenant resource. 409: valid user but conflicting workflow state, e.g. editing a signed/cancelled study.

Q: Why audit events?
A: Assignment, report and TAT changes record actor, action, entity and metadata in the same transactional workflow. This is useful traceability, but production audit requirements need stronger tamper resistance, retention and access controls.

8. Report workflow and allocation

Workflow: Unassigned → Assigned → Draft → optional Review Requested → Reviewed → QA Pending → QA Approved → Signed. QA can reject, returning it to QA Rejected for revision; cancelled studies cannot be reported. Scan scheduling/status is separate from report status.

Q: Who can save a draft?
A: Only the assigned active radiologist, for a performed, non-cancelled study in an editable report stage. The server enforces this, not just the worklist button.

Q: What is a second opinion?
A: The primary radiologist requests a different eligible radiologist. That reviewer submits review text; the primary still owns the draft and QA submission. The reviewer does not sign the primary report.

Q: What happens at QA and signing?
A: QA approval or rejection is restricted to the assigned QA user; rejection requires feedback. Only the assigned signer can sign a QA-approved report. Signing records UTC timestamps, marks report Signed and scan Completed. This is a workflow sign-off, not a legally certified digital signature.

Q: How does round robin work?
A: Candidates must be active radiologists eligible for the scan modality. A cursor keyed by tenant and modality stores the last chosen user; selection rotates through eligible IDs. PostgreSQL row locking protects the cursor during concurrent selection. SQLite test behavior is not a substitute for a PostgreSQL concurrency test.

Q: What is least-active assignment?
A: Count each eligible radiologist's active reports in the tenant, choose the smallest load, then use a stable tie-breaker. It is a simple demo heuristic; a real system might include shift, subspecialty, priority and capacity.

Q: Why are templates tenant-scoped?
A: An admin can create/archive modality-specific outlines for their organization. The radiologist can see only active matching-modality templates and must explicitly apply/review/save one. Starter templates contain neutral prompts, not prewritten diagnoses.

9. Time, TAT, MIS and migrations

Q: What is TAT?
A: Normally the minutes from performed_at to signed_at; legacy completed records may use reported_at. Pending performed studies show elapsed minutes against a modality target. A manual completed-study override requires a reason and does not rewrite event timestamps.

Q: Why store UTC?
A: It avoids ambiguous persisted local times and makes duration calculations consistent. The UI converts timestamps to the viewer's local timezone. Date filters are converted to UTC boundaries before comparison.

Q: How is MIS calculated?
A: The admin endpoint filters tenant studies (including modality, radiologist and performed-date range), computes per-study TAT and aggregates average, median, on-time percentage and overdue pending count. It currently materializes rows in Python; SQL aggregation/pagination would help with scale.

Q: How are schema changes deployed?
A: Fresh databases use init-db; existing databases use additive Flask CLI upgrade commands for tenancy, reporting, TAT, structured sections and templates. Before migrating existing data, back up the DB. These are hand-written migrations; moving to Alembic with versioned revisions would improve long-term maintainability.

10. Testing, deployment and honest limitations

Q: What tests exist?
A: Backend pytest tests exercise auth, reporting transitions, worklist, tenancy isolation and templates with isolated in-memory SQLite fixtures. The frontend has TypeScript build and Oxlint checks, but no dedicated automated UI test suite yet. Do not claim passing CI or browser tests unless you run them.

Q: Why can SQLite tests miss problems?
A: PostgreSQL differs in locking, constraints, timestamp semantics and query behavior. Use integration tests against PostgreSQL for migration, composite FK and concurrent round-robin confidence.

Q: How would you deploy?
A: Build static React assets, serve them over HTTPS, route /api to a production WSGI Flask deployment, configure cookie/CORS/CSRF rules, use a non-superuser PostgreSQL role, run migrations, manage secrets outside Git, and add monitoring/backups. The current Vite proxy and Flask development server are not production infrastructure.

Q: What are the biggest code-quality improvements?
A: Split App.tsx into routes/layout/feature hooks, remove leftover starter CSS, add frontend component/e2e tests, centralize query invalidation, validate API responses at runtime, add versioned migrations and production-grade tenant/security controls.

Q: If given one more week?
A: First make the repo reproducible and tested end to end; then add routing and focused UI tests around login, filters, worklist and reporting transitions. I would not add DICOM infrastructure before the foundations are solid.

11. Live demo script (3 minutes)

- Explain the two repos and /api flow. Sign in as admin; show role/tenant context, user management and report templates.
- Open patient search and calendar. Mention server pagination, URL query parameters and UTC-to-local display.
- Show assignment choices and why backend guards every operation. Sign in as radiologist; show only assigned studies in My worklist.
- Open a performed study. Apply a template, explain local unsaved sections, save a draft, then describe QA/review/signing without making a clinical claim.
- Show MIS/TAT and end with one limitation plus a realistic next step. If the demo data does not have an eligible study in a given state, describe the route/test instead of pretending the button worked.

12. Quick memory hooks

Vite builds/serves the UI; React renders it; TypeScript checks contracts; TanStack Query manages server state; Flask owns business rules; SQLAlchemy maps data; PostgreSQL persists it.

Authentication = who are you? Authorization = what may you see/do? Tenant scope = which organization's data? Workflow state = is this action valid now?

Cookie is HttpOnly; UI never handles raw JWT. /auth/me restores auth. Cache is cleared on user switch/logout. Server rules remain the security boundary.

UTC in database, local time in UI. TAT = performed-to-signed unless explicitly marked manual. DICOM viewing, production-grade PHI security, RLS, frontend test automation and certified signing are not part of this demo.